APPENDIX

VI. Detailed Results

A. Additional Policy Evaluation Results

1) Proof-of-Concept for Policy Evaluation on Push-T: We introduce the Push-T task as an additional proof-of-concept for policy evaluation [51]. The objective of this task is to push a T-shaped object to a target location and orientation. To streamline the task completion judgment, we use OpenCV to assess whether the T-shaped object aligns with the target pose in the final frame. The results are detailed in Table III, and the corresponding rollout videos are available on our project website.

TABLE III: **Policy Evaluation on Push-T** Pearson and MMRV scores (mean $\pm$ std) for Cosmos-Predict2-2B and IRASim.

Method	Pearson ↑	MMRV ↓
Cosmos-Predict2-2B	0.961 ± 0.026	0.042 ± 0.024
IRASim	0.610 ± 0.028	0.213 ± 0.030

2) Details for Benchmarking with Different Pre-trained Video Models: To better understand which video model performs best in a policy evaluation scenario, we evaluated several additional baselines:

- **IRASim** [20]: A diffusion transformer that processes 16 frames and includes a frame-level action-conditioning module within each transformer block.
- **Cosmos-Predict-1 (7B and 14B)** [17]: We fine-tuned the `Cosmos-Predict1-7B-Video2World` and `Cosmos-Predict1-14B-Video2World` models to perform next-frame prediction based on a single action. Following the methodology in their white paper, action conditioning is applied to the time embeddings of the model.
- **Wan 2.1 LoRA** [19]: We fine-tuned the `Wan 2.1 I2V-14B-480P` model using LoRA, applying action conditioning to the model's time embeddings.
- **Cosmos-Predict-2 (2B)** [18]: We fine-tuned `Cosmos-Predict2-2B-Video2World`, similarly applying action conditioning to the time embeddings.

The resulting correlation graphs are presented in Figure 9.

3) Real-world Experiments: For the real-world experiments, we collected an additional 300 trajectories within our specific setup to further narrow the domain gap between the standard Bridge dataset and our environment. Table VI reports the policy evaluation results both with and without these supplementary trajectories. The quantitative results demonstrate that including these local trajectories is critical for reliable policy evaluation.

4) Policy Evaluation with Human Evaluation: To validate our automated metrics, we conducted human evaluations alongside the VLM to determine whether each rollout was successfully completed. These results are presented in Table IV. Although the VLM annotations are not completely flawless (typically achieving 65-80% accuracy), we observed that while the Pearson coefficient is impacted, the MMRV metric remains highly stable. This stability indicates that the VLM is still a robust and useful component for annotating rollout videos. Furthermore, when comparing VLM annotations against human judgments, we found that the VLM is notably more accurate on shorter rollout videos, such as the Lift and Can tasks.

5) Failure Cases from the VLM: While the main paper primarily discusses policy evaluation failures caused by the video model, this section details failures stemming from VLM misjudgments. Despite carefully designing our prompts to enhance the reliability of the automatic annotation, occasional failures still arise due to inherent VLM limitations. We identified two major failure cases:

First, even though we explicitly instruct the VLM to ignore frames following the completion of a task, it sometimes fails to do so and incorrectly factors those later frames into its judgment (see Figure 10).

Second, because we uniformly sample frames from the rollout video for the VLM to evaluate, critical failures that occur during the un-sampled intervals can be missed entirely. This oversight leads the VLM to incorrectly judge a failed rollout as a success (see Figure 11).

B. Action-Conditional Video Prediction Results

We provide additional qualitative results for action-conditional video prediction on the RoboMimic dataset in Figure 12. Additionally, building on the Bridge dataset policy evaluation discussed in the main paper [53], [52], we present the corresponding action-conditional video prediction results in Table VII. For an extended set of rollout videos and results, please refer to our project website.

Finally, to investigate whether the video model suffers from accumulated error over time, we provide a quantitative comparison of video prediction methods. As shown in Table VIII, video prediction utilizing teacher forcing, where frames are generated conditioned on ground-truth previous frames rather than auto-regressively generated frames, consistently outperforms the standard auto-regressive approach.

VII. Design Space of Action-conditional Video Prediction

We conduct a series of experiments to explore the design space for adding action conditioning to an existing video model. This exploration covers network architecture, training strategies, and action representations. We use standard video prediction metrics to evaluate the resulting action-conditional models.

a) Architecture.: We investigate several architectural factors that potentially affect video prediction performance. First, we explore various approaches for incorporating action conditioning into diffusion models. Similar to prior work [55], one approach is to leverage the cross-attention mechanism [56] within DiT blocks to integrate action information (as shown in Figure 13(a)). Note that this modification does not alter the original network backbone, which helps preserve the prior information learned from Cosmos.

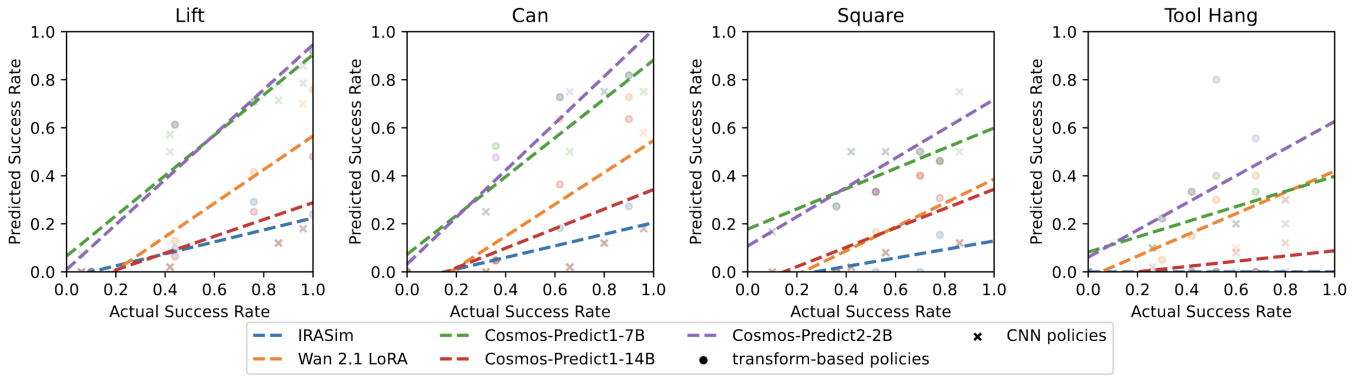


Fig. 9: **Correlation Plots for Policy Evaluation with Different Video Model**. In general, we found that Cosmos-Predict-2-2B perform the best.

TABLE IV: **Quantitative Results for Policy Evaluation in Simulation Environment with different task completion judgment**. Pearson correlation coefficient (higher is better) and MMRV (Mean Metric Rank Violation; lower is better) between video-model and simulator policy evaluations.

Task	**Lift**		**Can**		**Square**		**Tool Hang**	
	Pearson ↑	**MMRV ↓**	**Pearson ↑**	**MMRV ↓**	**Pearson ↑**	**MMRV ↓**	**Pearson ↑**	**MMRV ↓**
Cosmos-Predict-2-2B (VLM)	0.879	0.015	0.801	0.146	0.810	0.168	0.790	0.222
Cosmos-Predict-2-2B (Human Eval)	**0.888**	**0.014**	**0.860**	**0.141**	**0.849**	**0.165**	**0.833**	**0.217**

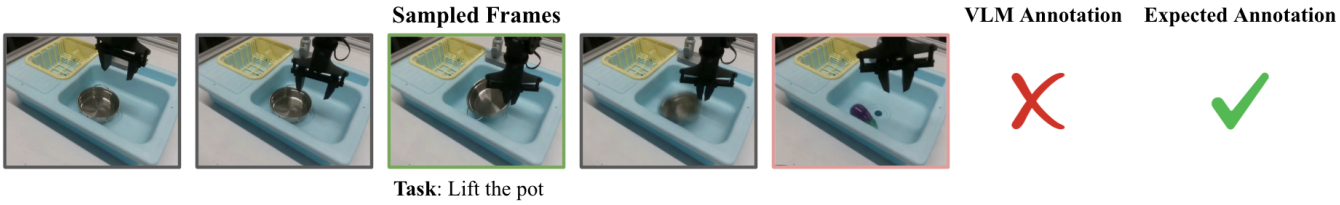


Fig. 10: Hallucination can occur after task completion. According to our prompt design, the VLM is instructed to ignore frames following the completion of the task; however, it sometimes fails to do so and includes those frames in its judgment.

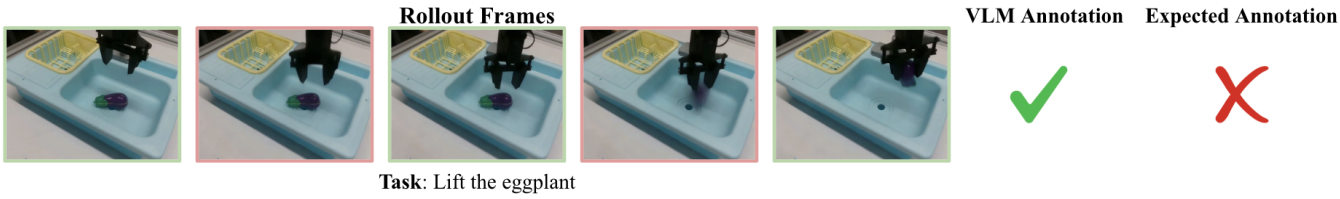


Fig. 11: Hallucination can occur in the skipped frames, so VLM skips it and incorrectly judge the rollout video as success.

TABLE V: **VLM Annotation Accuracy**. The VLM achieves higher annotation success rates on short-horizon tasks and lower rates on long-horizon tasks.

Model	**Lift**	**Can**	**Square**	**Tool Hang**
VLM Accuracy	0.79	0.71	0.68	0.66

TABLE VI: **Real-World Policy Evaluation.** Pearson and MMRV scores (mean ± std) for Cosmos and IRASim.

Method	**Pearson ↑**	**MMRV ↓**
Cosmos	0.687 ± 0.031	0.170 ± 0.024
Cosmos w/o trajectories in our bridge	0.611 ± 0.019	0.209 ± 0.022
IRASim	0.610 ± 0.028	0.213 ± 0.030

Another option is to append the action token to the image token along the channel dimension (as shown in Figure 13(b)). This strategy requires modifying the first DiT block in Cosmos, which could potentially disrupt the prior knowledge the model previously acquired. These architectures are illustrated in Figure 13, and the quantitative results are summarized in Table IX. Interestingly, while most prior works condition diffusion models using cross-attention, our results reveal that integrating actions through time embeddings yields

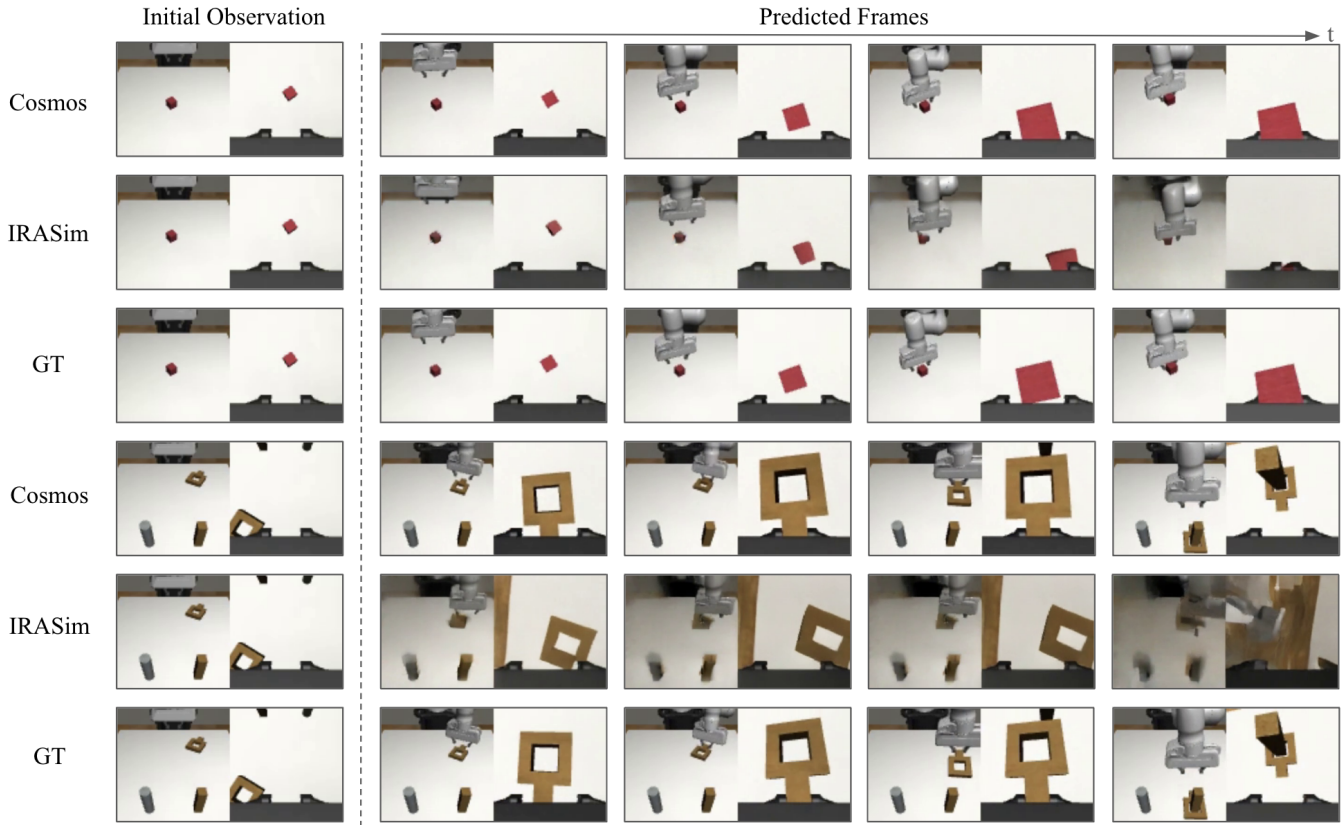


Fig. 12: **Action-conditional Video Prediction Results**. The predicted results of Cosmos are closed to the ground-truth results.

TABLE VII: Evaluation of action-based video prediction on Bridge dataset.

Method	PSNR ↑	SSIM ↑	Latent L2 ↓	FVD ↓
IRASim [20]	19.13	0.64	0.38	593
Cosmos [17]	**21.14**	**0.82**	**0.32**	**190**

the best overall performance.

b) Finetuned or Frozen Backbone.: In our primary experiments, the entire network is optimized in an end-to-end manner. While this approach is time-consuming and requires substantial GPU memory, it proves to be necessary. An alternative strategy is to freeze the pre-trained backbone and fine-tune only the action encoder to reduce GPU memory consumption and improve training efficiency. However, our empirical results indicate that this frozen-backbone strategy leads to suboptimal performance.

c) Action Representation.: Because raw actions often manifest as low-frequency signals, we apply positional embeddings to project them into a high-frequency representation. Table IX also includes results from a baseline model trained without positional embeddings. The data demonstrates a clear drop in performance, confirming the necessity of this high-frequency projection step.

VIII. Experimental Details

A. Simulation Experimental Setup

1) Action-conditional Video Prediction Model:

a) Training and Evaluation Configuration.: Our action-conditional video model is trained for 240K steps with a batch size of 128 and a learning rate of 4e-4. We utilize 32 H100 GPUs to ensure efficient large-scale training. We split the RoboMimic [46] dataset into training, validation, and testing sets using an 8:1:1 ratio.

For video prediction evaluation on the RoboMimic dataset, we provide a single conditional frame and autoregressively generate 64 future frames, which are then compared against the ground-truth frames. For the Bridge dataset [53], [52], where the variance in trajectory length is significantly smaller than in RoboMimic, we condition the model on the initial frame and autoregressively generate frames until the end of the trajectory.

b) Policy Rollout Dataset.: In addition to utilizing human demonstrations from the RoboMimic dataset [47], [46], we generate synthetic rollouts using trained policies to further augment the training data for our action-conditional video prediction model. For each task, we train a Diffusion Policy on the RoboMimic dataset to acquire four distinct policies of varying proficiency. The specific success rates for these policies are as follows:

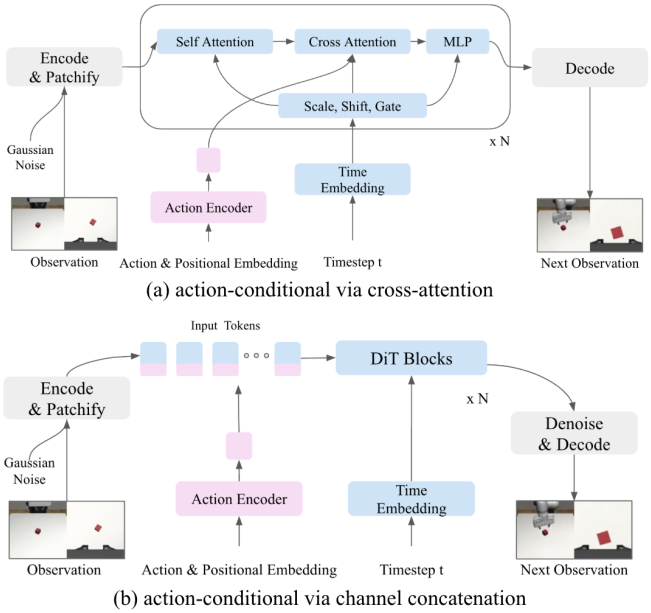
- **Lift**: 0.04, 0.38, 0.82, 1.00

TABLE VIII: **Comparison between Auto-Regressive Generation and Teacher-Forcing in Action-Conditional Video Prediction.** We generate 64 future frames in an auto-regressive manner and evaluate the models on both RoboMimic (in-distribution) and policy rollout (out-of-distribution) trajectories.

World Model	RoboMimic (In-Distribution)				Policy Rollouts (Out-of-Distribution)			
	PSNR ↑	SSIM ↑	Latent L2 ↓	FVD ↓	PSNR ↑	SSIM ↑	Latent L2 ↓	FVD ↓
Cosmos-Predict2-2B (Auto-Regressive)	20.622	0.868	0.315	98.455	15.996	0.793	0.458	250.234
Cosmos-Predict2-2B (Teacher Forcing)	**22.311**	**0.899**	**0.133**	**71.455**	**20.122**	**0.813**	**0.378**	**120.770**

TABLE IX: **Ablation Study** on making Cosmos action-conditional on the Robomimic dataset. We investigate different model architectures, training strategy and action representations.

Method	PSNR↑	SSIM↑	Latent L2↓	FVD↓
Cosmos via Time-Embedding	**20.622**	**0.868**	**0.315**	**98.455**
Cosmos via Channel-Concat	18.671	0.826	0.397	154.816
Cosmos via Cross-Attention	19.764	0.843	0.372	165.547
Cosmos w/ Frozen Backbone	13.012	0.700	0.711	661.012
Cosmos w/o Positional Embedding	20.012	0.841	0.391	128.123



(a) action-conditional via cross-attention

(b) action-conditional via channel concatenation

Fig. 13: **Design space for action-conditional next frame prediction**. Alternatively, we added action conditioning through (a) channel dimension (b) cross attention.

- **Can**: 0.00, 0.38, 0.56, 0.84
- **Square**: 0.36, 0.64, 0.74, 0.78
- **Tool Hang**: 0.26, 0.54, 0.64, 0.66

2) Policy Evaluation:

a) Policies.: We adopt the Diffusion Policy framework [51] alongside the RoboMimic dataset to train a comprehensive suite of policies. For each task, we evaluate four policies implemented with both Transformer and CNN backbones. Performance is measured by rolling out each policy in a simulation environment across 50 randomized initial configurations. The resulting success rates for each task are detailed below:

TABLE X: **Dataset Statistics.** Each cell reports the number of trajectories / transitions for Proficient-Human (PH), Multi-Human (MH), and Policy Rollouts across different tasks.

Task	PH	MH	Policy Rollout
Lift	200 / 9k	300 / 31k	400 / 160k
Can	200 / 23k	300 / 62k	400 / 160k
Square	200 / 30k	300 / 80k	400 / 160k
Tool Hang	200 / 95k	– / –	400 / 280k

- **Lift**: Transformer (0.10, 0.44, 0.76, 1.00), CNN (0.06, 0.42, 0.80, 0.94)
- **Can**: Transformer (0.00, 0.36, 0.62, 0.90), CNN (0.32, 0.66, 0.80, 0.96)
- **Square**: Transformer (0.26, 0.52, 0.70, 0.80), CNN (0.10, 0.46, 0.80, 0.96)
- **Tool Hang**: Transformer (0.30, 0.42, 0.52, 0.68), CNN (0.00, 0.26, 0.60, 0.80)

B. Real-World Experimental Setup

1) Action-conditional Video Prediction Model: For real-world tasks, our action-conditional video model is trained for 200K steps with a batch size of 128 and a learning rate of 4e-4, again utilizing 32 H100 GPUs for efficient training. For the Bridge tasks, we use the dataset configuration processed by IRASim [20].

2) Policy Evaluation:

a) Policies.: The evaluated generalist policies include Octo-Small, Octo-Base [3], and OpenVLA [4]. To assess real-world performance accurately, we replicate the standard Bridge physical setup [53], [52] and execute each policy over 20 independent trials to measure its success rate. These quantitative results are summarized in Table XI.

TABLE XI: Performance comparison across different objects and models.

Model	Eggplant	Pot	Carrot	Cup
OpenVLA	0.30	0.50	0.50	0.70
Octo_Small	0.10	0.30	0.15	0.20
Octo_Base	0.15	0.35	0.10	0.30

IX. Auto-Annotation for Rollout Videos with VLMs

A. Prompt Design

To reduce the manual effort required to annotate task completion in rollout videos generated by our models and policies, we leverage Gemini-2.5-Pro [22] to automate the evaluation process.

The prompt structure designed for Gemini-2.5-Pro comprises three main components:

- **system_prompt**: Specifies the assistant's behavior, role, and persona.
- **begin_prompt**: Marks the start of the interaction and provides the task description alongside specific evaluation criteria.
- **end_prompt**: Signals the end of the input frames and prompts the model to generate its final judgment.

The complete API query sent to Gemini-2.5-Pro concatenates the system prompt, the begin prompt, the set of uniformly sampled image frames, and the end prompt. Below, we detail the specific prompts utilized for each task in our experiments.

1) Prompts for Simulation Experiments: For our simulation experiments, we evaluate four different tasks. The system prompt and end prompt remain consistent across all four tasks and are defined as follows:

```
system_prompt = "You are an expert in judging
     whether a robot arm performs a given
    task successfully. The demonstration
    video contains two views: one is the
    robot arm's view, and the other is the
    global human's view. You are responsible
    for providing a task completion judgement
    , given image frames uniformly sampled
    from the demonstration video."

end_prompt = "Please judge whether the robot
    arm performs the task correctly with a
    binary label (Yes or No)."
```

The specific `begin_prompt` for each task is detailed below.

Lift.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The cube is successfully lifted by the
    robot arm, which means the gripper is
    closed around the cube and the cube is
    moved upwards.
2. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The information from the human's view is
    not consistent with the robot arm's view.
2. The video is not clear or of good quality.
     For instance, the boundaries of the
    object or robot arm are not sharp and
    clear.
3. The cube is not always visible in the
    video.
4. Shot changes occur. For instance, the
    viewpoint and the background must be
    consistent between frames.
5. There is distortion or blurring that makes
     it difficult to see the task.
6. There is no red cube in the video.
7. The task is not complete.
"
```

Can.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The arm closes around the coke can and
    moves it consistently with the robot arm'
    s motion.
2. The coke can is lifted and moved to the
    bin.
3. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The information from the human's view is
    not consistent with the robot arm's view.
2. The video is not clear or of good quality.
     For instance, the boundaries of the
    object or robot arm are not sharp and
    clear.
3. The coke can is not always visible in the
    video.
4. Shot changes occur. For instance, the
    viewpoint and the background must be
    consistent between frames.
5. There is distortion or blurring that makes
     it difficult to see the task.
6. There is no coke can in the video.
7. The task is not complete.
"
```

Square.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
```

```
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The square nut is successfully lifted by
    the robot arm, which means the gripper is
     closed around the square and the square
    is moved upwards.
2. The square nut is moved consistently with
    the movement of the end-effector of the
    robot arm.
3. The square nut is placed onto a rod at the
     end.
4. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The robot arm does not complete the task.
2. The information from the human's view is
    not consistent with the robot arm's view.
3. The video is not clear or of good quality.
     For instance, the boundaries of the
    square nut or robot arm are not sharp and
     clear.
4. The square nut or rod is not always
    visible in the video.
5. Shot changes occur. For instance, the
    viewpoint and the background must be
    consistent between frames.
6. The square nut or rod is missing from the
    video.
7. The square nut or rod moves when the arm
    has no contact with it.
"
```

Tool Hang.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The robot arm must assemble a frame by
    inserting a hook piece into a base.
2. The arm hangs the wrench on the hook.
3. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The robot arm does not complete the task.
2. The information from the human's view is
    not consistent with the robot arm's view.
3. The video is not clear or the quality is
    poor.
4. Shot changes occur. For instance, the
    viewpoint and the background must be
    consistent between frames.
5. The hook or wrench is not visible in the
    video.
6. The wrench moves when the arm has no
    contact with it.
"
```

2) Prompts for Real-World Experiments: For our real-world experiments, we evaluate four distinct tasks. The system prompt and end prompt remain the same for all four real-world tasks and are defined as follows:

```
system_prompt = "You are an expert in judging
     whether a robot arm performs a given
    task successfully. The demonstration
    video contains a black robot arm with a
    gripper and a set of toy objects. You are
     responsible for providing a task
    completion judgement, given image frames
    uniformly sampled from the demonstration
    video."

end_prompt = "Please judge whether the robot
    arm performs the task correctly with a
    binary label (Yes or No)."
```

The specific `begin_prompt` for each real-world task is detailed below.

Lift the eggplant.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The gripper closes around the eggplant.
2. The eggplant is lifted by the gripper.
3. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The robot arm fails to complete the task.
2. The video is unclear or of poor quality.
3. Shot changes are present. For example,
    inconsistencies in viewpoint or
    background between frames.
4. The object moves without any contact from
    the arm or gripper.
5. The object's appearance (e.g., shape or
    color) changes during the video.
6. The eggplant turns into a carrot.
"
```

Lift the pot.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The gripper closes around the pot.
2. The pot is lifted by the gripper.
3. Once the task is completed, please ignore
    the rest of the video.
```

```
Failure criteria:
1. The robot arm fails to complete the task.
2. The video is unclear or of poor quality.
3. Shot changes are present. For example,
    inconsistencies in viewpoint or
    background between frames.
4. The object moves without any contact from
    the arm or gripper.
5. The object's appearance (e.g., shape or
    color) changes during the video.
6. The size of the object changes.
"
```

Lift the carrot.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The gripper closes around the carrot.
2. The carrot is lifted by the gripper.
3. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The robot arm fails to complete the task.
2. The video is unclear or of poor quality.
3. Shot changes are present. For example,
    inconsistencies in viewpoint or
    background between frames.
4. The object moves without any contact from
    the arm or gripper.
5. The object's appearance (e.g., shape or
    color) changes during the video.
6. The size of the object changes.
7. The carrot turns into an eggplant.
"
```

Lift the cup.

```
begin_prompt = "
Please judge whether the robot arm performs
    the task correctly with a binary label (
    Yes or No) based on the provided success
    and failure criteria. The task is
    successful if it satisfies all success
    criteria and fails if it meets any
    failure criteria.

Success criteria:
1. The gripper closes around the pink cup.
2. The pink cup is lifted by the gripper.
3. Once the task is completed, please ignore
    the rest of the video.

Failure criteria:
1. The robot arm fails to complete the task.
2. The video is unclear or of poor quality.
3. Shot changes are present. For example,
    inconsistencies in viewpoint or
    background between frames.
4. The object moves without any contact from
    the arm or gripper.
5. The object's appearance (e.g., shape or
    color) changes during the video.
6. The handle of the cup disappears.
"
```